

10. Overcoming the Challenges in Quality Assurance

Hugo Filipe Oliveira Rocha¹

(1) Ermesinde, Portugal

This chapter covers:

- Understanding the different kinds of tests and their scope in the overall architecture
- The challenges of guaranteeing the quality of the end-to-end flow in event-driven services
- Using contract testing and consumer-driven contracts as core concepts in the quality assurance process
- Strategies to safely validate functionality in the production environment

As systems evolved from overly complex single units to constellations of highly distributed smaller components, in many ways and in many businesses, quality assurance seems to be left behind. It's peculiar how we evolved our architecture designs, development processes, and deployment strategies to accommodate the paradigm shift of highly scalable distributed architectures, but the practices of quality assurance remain roughly the same.

We came a long way from the manual validations and lengthy regression testing of traditional applications and embraced, as we should, the joys of automated testing. However, to guarantee the correctness of the end-to-end flow as the end user sees it, we often rely on the same approaches we always did, by deploying all functionality in a quality environment and running manual or automated validations against the whole system. There is a mismatch between this approach and the very

core principles of distributed microservice architectures and of independent deployment procedures and autonomous release cycles.

Despite the other kind of difficulties and limitations, monoliths provide the means to easily perform end-to-end validations. By having a single deployable unit, it's often possible (although not always easy) to deploy the application and its dependencies to test the end-to-end flow. When dealing with an architecture composed of dozens or even hundreds of independent components, it easily gets extremely complex to perform pre-production validations against the whole end-to-end flow. The practices of DevOps and continuous delivery promote a fast feature delivery flow in order to provide fast feedback. The autonomous release cycle of each independent service in microservice architectures is a way to achieve it. However, how are we able to accomplish independent deployments of each component while validating the integrity of the end-to-end flow?

The typical test categories ranging from unit to integration tests still apply to event-driven architectures. They provide higher decoupling characteristics which we can combine to achieve the more sustainable approaches we discuss in this chapter. Depending on the use case and the conditions of your company, you might still need to invest in complex end-to-end tests; however, as we will discuss at the end of the chapter, we will challenge you to adopt more sustainable and scalable approaches by combining a pre-production quality assurance process with safe production validations.

10.1 Microservice Testing Approaches and How They Relate to Event-Driven

There's a lot of confusion in the definition of the several types of tests. Integration, component, and end-to-end tests are often used interchangeably and change their meaning depending on the context. In this section, we will explain the different categories of tests, their scope, and how they apply to event-driven services.

There are several categories of tests in software engineering; you most likely came across several of them while developing new functionality. In the past, manual testing was a substantial step of the quality assurance process, although thankfully most businesses have been focusing on a more automation mindset. Most types of tests still apply to event-driven services, the same as they would in a traditional application or a monolith.

Event-driven architectures often imply a considerable number of components and frequent releases. As we will discuss in the next section, a strong automation mindset is pivotal for the success of these systems. Manual testing is still useful, but rather than having people doing a suite of repetitive tasks on every release, test automation frees them to do more meaningful work by interacting with the application in unexpected or creative ways (through, for example, exploratory testing).

10.1.1 Unit Tests

Unit tests validate functionality on a section of code inside a service. Typically, in the context of a function or class, there isn't any hard rule on how small they should be. Unit tests should be present throughout the several layers of the service's architecture. You might recall when we discussed the event-driven service's architecture in Chapter [3](#) and worked through the N-tier and clean architecture. Each service typically has several layers with dedicated responsibilities. Each of these layers will have unit tests to validate the layer's functionality on the smallest possible scope. Figure [10-1](#) illustrates how unit tests fit in the overall event-driven architecture.

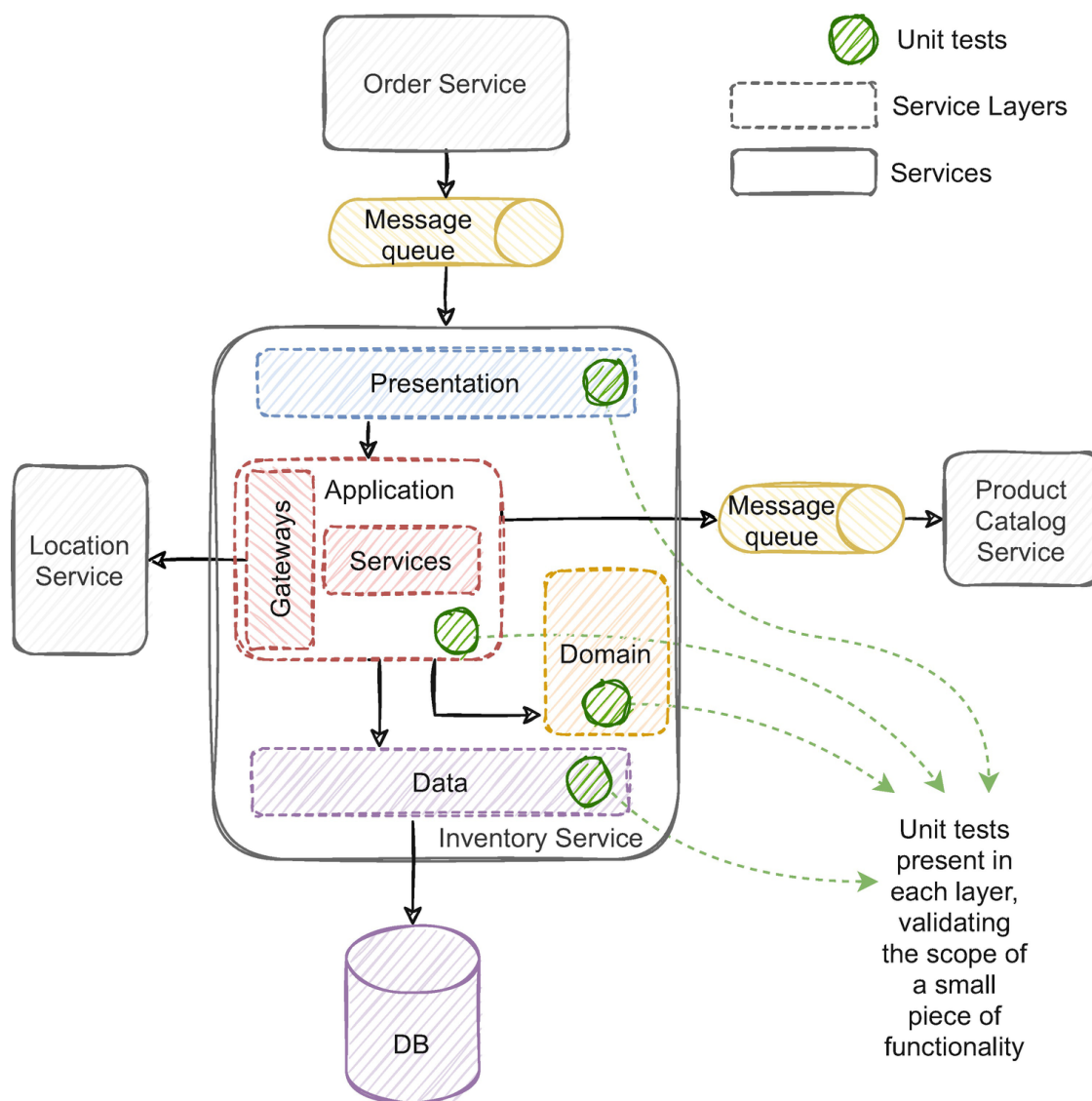


Figure 10-1 Unit tests on an event-driven service

Figure [10-1](#) illustrates the inventory service with several layers. It consumes events from the order service and publishes events to the product catalog service. It also does a synchronous request to the location service. Unit tests sit on each of the service layers validating solely the layer's functionality. If the piece of code the test is validating has a dependency on an external resource, like the database or external service or even another layer, the test mocks that request and focuses solely on the functionality implemented by that piece of code.

Although unit tests are fast and usually performant, it doesn't mean they are free. A pitfall I often see companies with strong unit testing culture fall into is creating unit tests for every line of code, whether it makes sense or not. Remember unit tests are code and have a maintenance cost; we need to maintain them. Code with hardly any logic likely doesn't need a test. People often measure quality by test code coverage; although it can

be an indicator (among many) alone, it doesn't mean much, and struggling to have 100% code coverage won't benefit you much. Instead, it might hinder your velocity due to the maintenance overhead. Be pragmatic, measure the cost, and understand if the test makes sense.

Having said that, unit tests are the fastest feedback loop we have access to, they usually are extremely fast to run, and we can expect applications to have large quantities of them. They are extremely useful to validate small scoped functionality. They also are good as an indicator of the code organization; if a unit test is considerably large or has extensive responsibility, it's often an indicator the code might need to be refactored and divided.

10.1.2 Component Tests

Unit tests are great to validate localized and small scoped functionality inside a service layer but provide no guarantees on how the several layers interact with each other to compose the overall service features. Component tests fill that gap; they validate the full code flow inside a service, going through every layer, but only in that service's scope, removing external dependencies.

Component tests don't exert external dependencies like the database, external services, or message brokers; they focus only on the functionality inside the boundaries of the service. You can see it as testing anything the service does as a single process, without running anything else. Figure [10-2](#) illustrates the component tests scope.

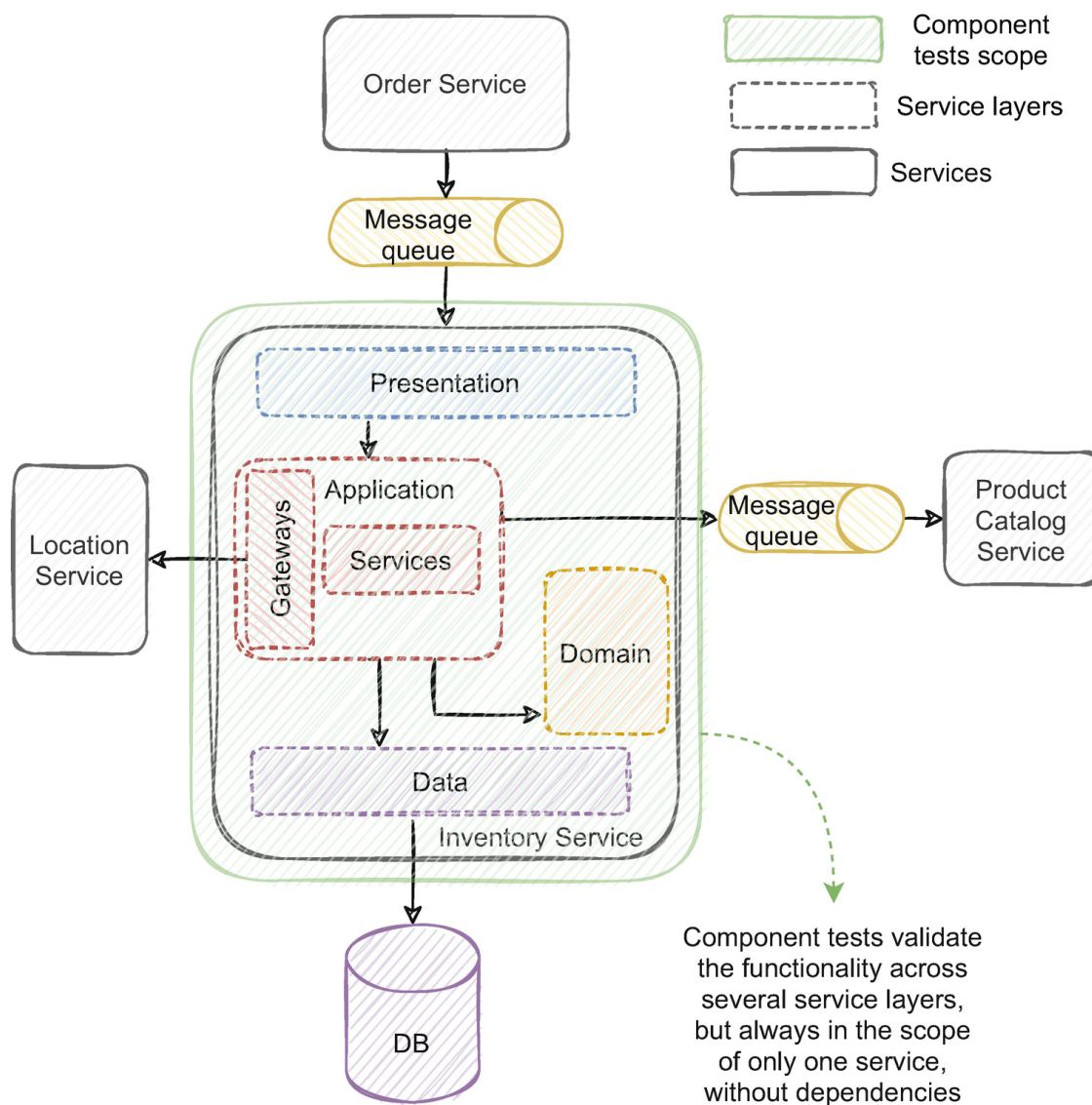


Figure 10-2 Component tests scope in an event-driven service

In this case, suppose the inventory service reacts to order created events to process stock, saves state in the database, and publishes a stock changed event. A component test for this flow could be the full flow of the order event processing without using the database, the message broker, or the location service.

A way to easily abstract the external dependencies is using dependency injection. While running the tests, the service can inject mock implementations and fake the normal or abnormal responses of external dependencies validating the correctness of the service.

Component tests are useful to validate the soundness of the sequence of steps or overall workflow of the service in a way unit tests aren't supposed to due to their limited scope. Well implemented, they can also be

extremely fast due to the lack of direct dependencies; everything can be run in memory.

However, since they abstract the dependencies to the external resources, they don't prevent bugs in their direct implementation. For example, if the inventory service is using a SQL database, we could write a component test to validate the order created event processing logic, but if we had an issue in the SQL query, the test wouldn't likely be able to detect it since the repository would be mocked.

10.1.3 Extended Component Tests

A way to mitigate the risk of issues in the implementation of the external dependencies (like the example of the SQL query we just discussed) is to use an extended version of the component tests. Instead of completely removing the external dependencies, we can use the in-memory version of those dependencies. For example, if the service has a database, we can use an in-memory implementation of the database. It can behave the same way a database would, and it would allow validating the soundness of the query.

Figure [10-3](#) illustrates the same use case but with in-memory components of the external dependencies.

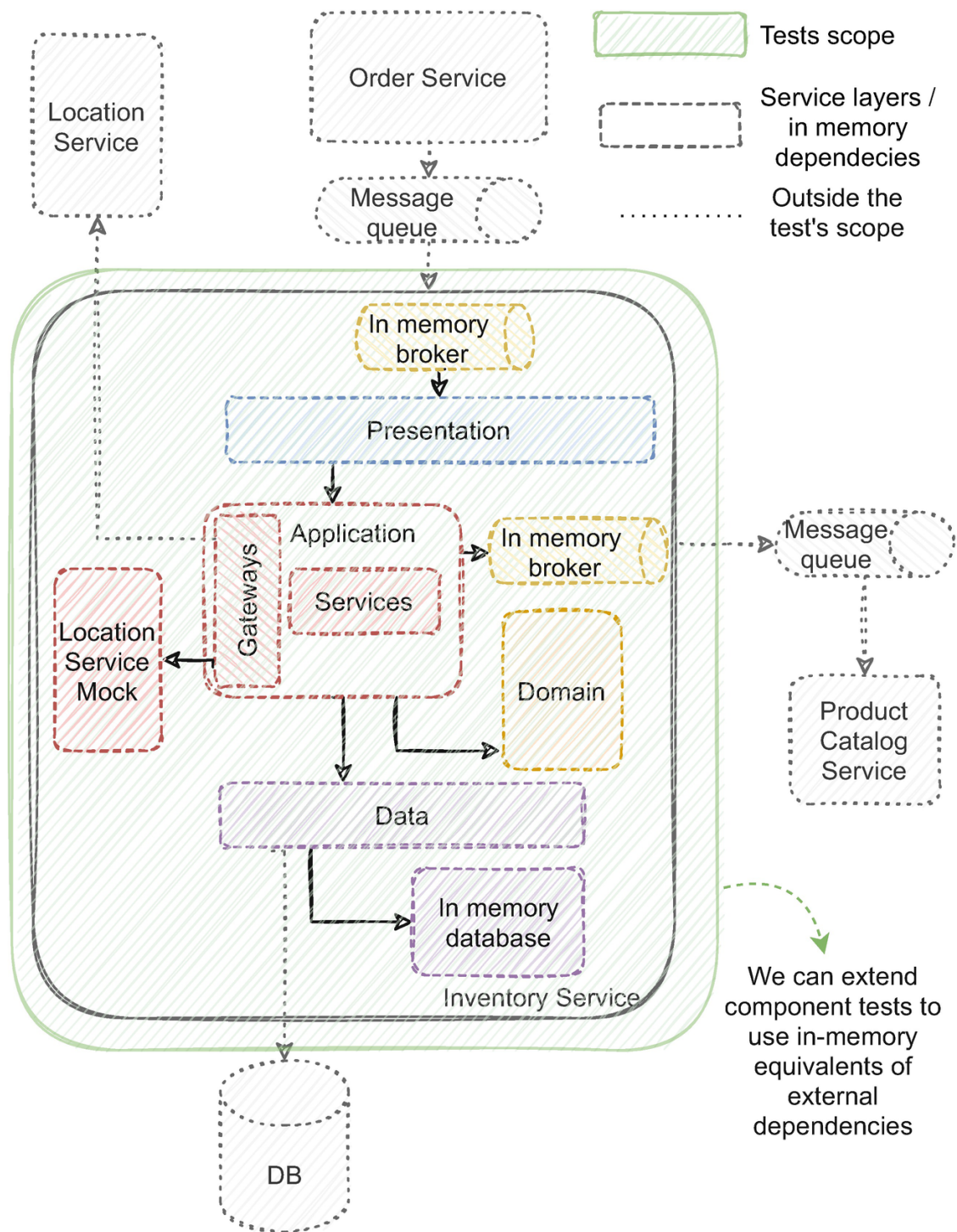


Figure 10-3 Extended component tests with in-memory dependencies

As you can see in the example, the external dependencies were replaced by their in-memory equivalent. The exception is the location service; it likely won't have an in-memory equivalent available since it is a service built in-house, but we can replace it with a mock or a stub. Extended component tests can paint a comprehensive scenario of the service and provide exceptionally valuable validations while not suffering from the drawbacks of managing external dependencies. They are able to validate the interactions with external dependencies while still being able to run extremely fast. The lack of external dependencies also facilitates

the setup, running the tests locally, and simplifies the release process; since the release pipeline won't need to manage the dependencies, only the service and the tests.

If they sound too good to be true, it's perhaps they often are. In theory, it's a great setup, but implementing them in practice is often hard due to the inexistence of the in-memory replacements of the external dependencies. There are some providers of in-memory database equivalents, for example, Microsoft's in-memory entity framework provider,¹ that can provide similar features of a real database without using external components. However, these providers often fall short of the actual, real-life implementations. Often implementations don't fully match the full features of a real database. Transactions, the ACID guarantees, aren't real when run in memory; everything works instantaneously and without the restraints of a real database.² H2 database engine ³ is also an interesting alternative to emulate databases in memory; however, it suffers from similar limitations and doesn't provide compatibility to all functionality. What people often end up doing is having an integration test against a real database validating the same thing, which raises the question of why to have the extended component test in the first place.

If you are using Java, Kafka has an embedded Kafka implementation which is a very interesting in-memory equivalent. Since both Kafka and ZooKeeper (for older Kafka versions) are Java applications, it is possible to run them from Java code. However, it doesn't reflect all implications of a real Kafka cluster.⁴ Elasticsearch also had an embedded alternative that could be used for testing; however, it was discontinued due to the difficulties of maintaining compatibility with new features (in this case security-related⁵).

A very promising alternative is using Testcontainers.⁶ They can provide an easy way for tests to use and run containerized instances of most databases. RabbitMQ, Kafka, and most message brokers provide containers that are compatible with this approach. Although they aren't actually component tests but are perhaps halfway between integration and component tests (half-breed integration tests as we will discuss in Subsection [10.1.5](#)), they can provide a reliable way to set up real dependencies with

real-life scenarios. Unlike the H2 database engine, they provide full database compatibility,⁷ since the actual database runs in a container.

For most use cases we've seen, teams tend to struggle to find reliable alternatives for external dependencies. But it really depends on the dependency; some of them might have valuable in-memory alternatives which we can use. They can complement integration tests well by limiting the number of integration tests and move most validations to faster and more reliable component tests. But most often than not, the alternatives are limited and can be used on somewhat niche or simple use cases.

10.1.4 Integration Tests

The meaning of integration tests has a lot of ambiguity, different references mention different scopes, and there's usually some confusion between component, integration, and end-to-end tests. We will define them with arguably the most consensual definition; tests designed to validate the communication and relationship with external dependencies. The tests validate the actual interaction with external components like databases, message brokers, other services, distributed caches, etc. We can approach them in two ways, on a smaller scope by validating only the interaction with each external dependency and nothing else or on a broader scope by validating the full business flow including the external components.

In the context of event-driven services, they usually include the interaction with the message broker but not with the service publishing messages. However, they usually include services with a synchronous dependency like an HTTP request. Event-driven services are decoupled by the event broker so it is a sound decision to relinquish other services in the message flow to the broader end-to-end tests. The consequence of direct synchronous requests between two services is higher coupling; thus, the requesting service can be directly affected by an issue with the target service. Hence, integration tests often include synchronous dependencies in their scope. Figure [10-4](#) illustrates the scope of integration tests.

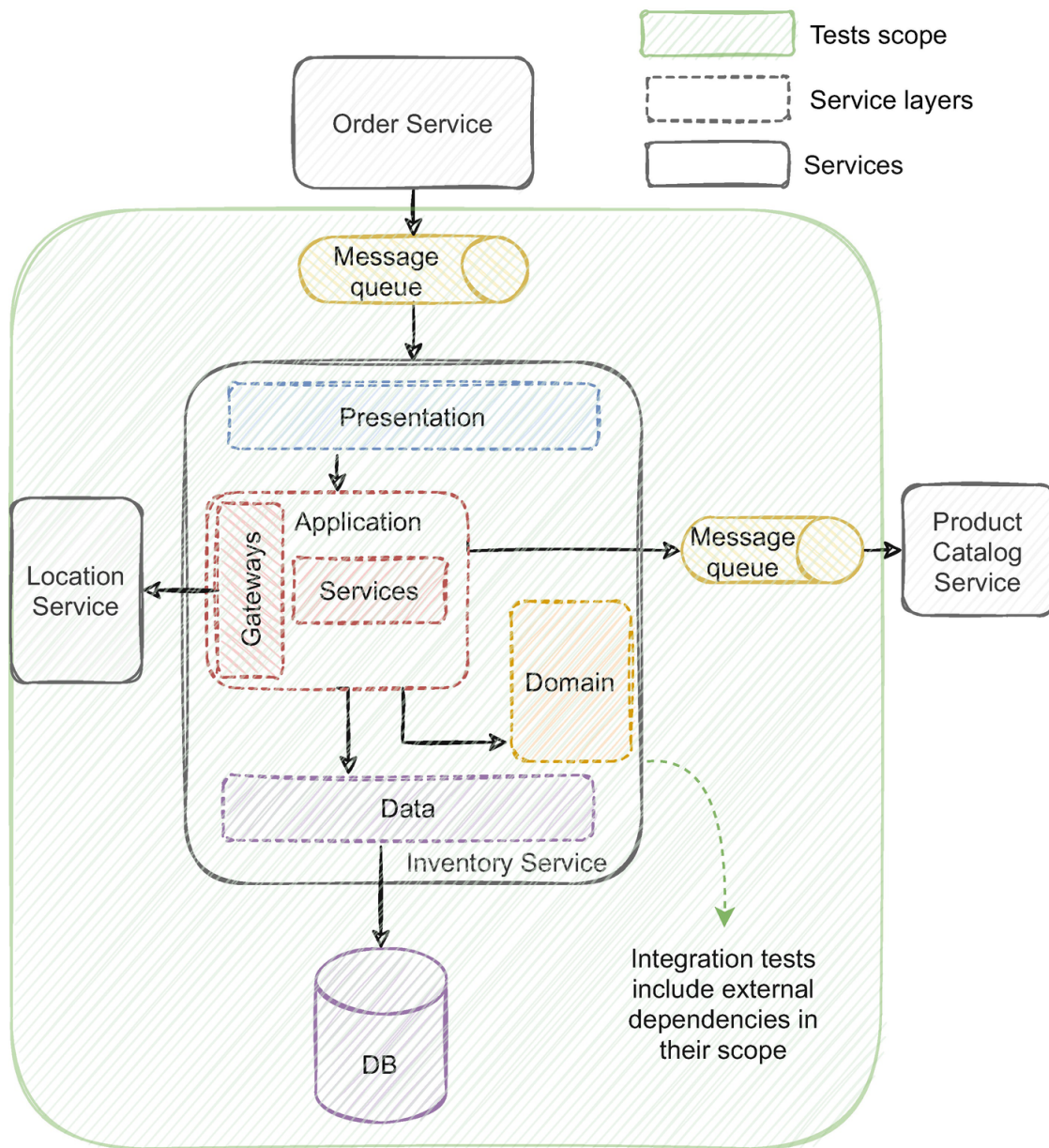


Figure 10-4 Scope of integration tests

These tests are often slower and more brittle than the ones we discussed until now. We need to guarantee there are the right conditions for the execution of the tests, the database needs to have the relevant data, caches need to be cleared, the existing messages in the broker must not affect the result of the tests, and so on. Guaranteeing these kinds of conditions on a daily basis can be complex to manage and often originate false positives. Containers can help with this; if we run containers and then remove them on a per test or per test session basis, it can help with these issues.

The direct dependency on the location service can be even harder to tackle. We need to guarantee we have a valid live version of the location service with relevant data for our test cases. If the location service has

several dependencies, it can be even harder to manage since we have to guarantee each dependency is in a valid condition for the service to respond. Guaranteeing all of these conditions is no small feat and raises several questions on how to release the inventory and the location service. Should we run these tests in every release of both services? We will further discuss these concerns in Section [10.3](#).

10.1.5 Half-Breed Integration Tests

There's a distinction between the service's mainstream dependencies, like databases or message brokers, and synchronous dependencies to other services, for example, the dependency between the inventory and location service. Mainstream dependencies are typically more stable and predictable. We aren't continuously changing our databases or message broker's version; it remains stable for a considerable amount of time, and when we change them, it often is a carefully scheduled operation.

However, other services on the architecture are being developed by fellow teams in the organization which might deploy their services frequently. Each has its own peculiarities in schema, data, configurations, and dependencies. Setting up a test to validate the interaction between two or several services can be exceptionally challenging due to the need to get everything right plus the need to maintain the quality environment synced with the newly released versions.

A variation of integration tests that ease that process is to delegate the responsibility to integrate other services in the ecosystem to the realm of end-to-end tests. Half-breed integration tests mock or stub external service dependencies but maintain mainstream dependencies like databases or message brokers. They can validate the service's functionalities on a broader scope and be more reliable since these dependencies are often more predictable and easier to set up with containers. Figure [10-5](#) illustrates this scenario.

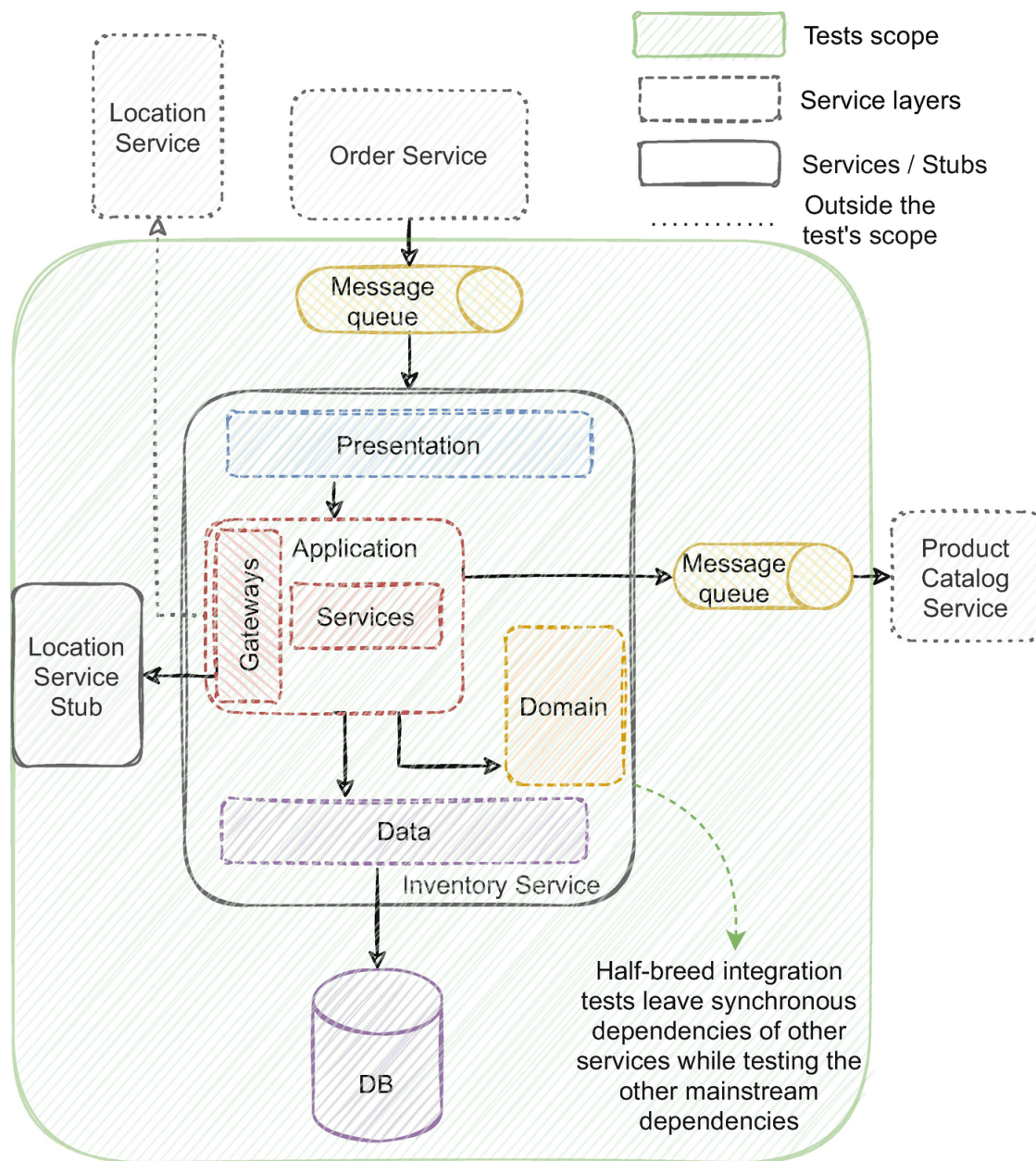


Figure 10-5 Scope of half-breed integration tests

These tests still need to maintain valid external components and guarantee the coherency of the data and message queues. However, it's only in the service's direct ownership, which greatly simplifies the test's development for the team governing the service. They don't need to worry about other services outside their ownership.

The tests still validate the interaction with the location service, but by mocking the request. In this case, using mocks is a good approach to simplify the overall setup while guaranteeing the correctness of some interactions between the two services. We no longer need to worry about deploying and maintaining a correct configuration and relevant data on the location service; we simply mock the calls we need to validate the flow of

the inventory service. However, mocks suffer heavily from confirmation bias. The teams will develop the mock's expectations according to what they believe the service will return, which can be troublesome when those assumptions are, or become, wrong. Either way, it's a great way to complement unit and component tests in a broader scope and guarantee the correctness of the overall flow of the service.

An interesting way I observed teams implementing them is having the tests bootstrap all the required preconditions for the tests and validating the end result. Figure 10-6 illustrates this approach with the inventory service handling of order created events.

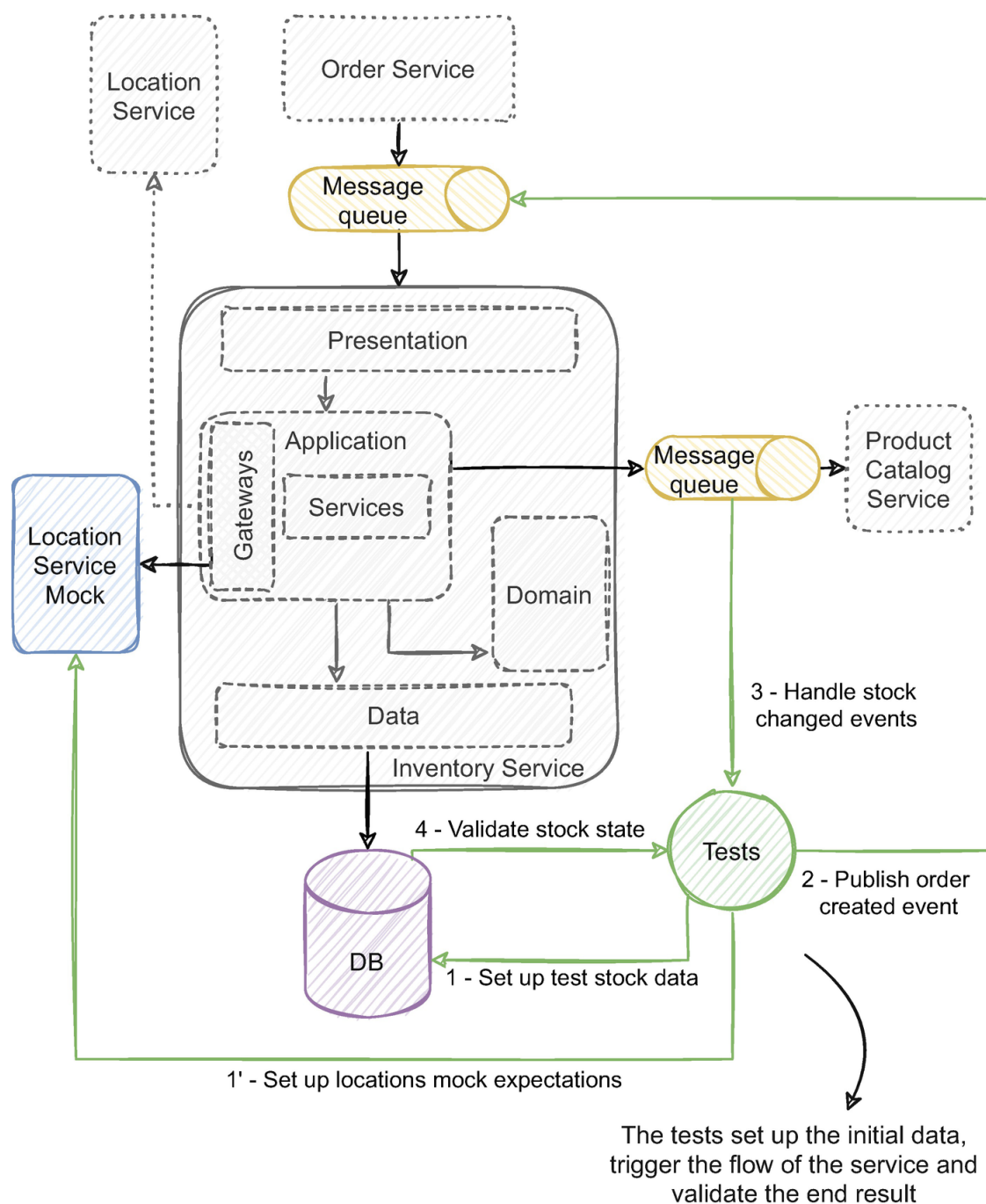


Figure 10-6 Example of half-breed integration tests with the order created event flow in the inventory service

Before running the actual tests, we bootstrap the inventory database by setting up data. For example, if the test case is when a new order is created, we decrease the stock quantity by one; we might need to set up the existing stock quantity before actually running the test. We also set up the expectations in the location service mock.

To trigger the start of the test, we publish an order created event to the inventory message queue. The service will handle the event and produce a set of expected results. For example, the service might decrease the stock and publish a stock changed event. We can then wait for the stock changed event by consuming the inventory service event queue. Once we receive the event, which signals the processing finished, we can validate the event's data and the state in the database. The main difficulty is to know when to stop waiting for the event; some test cases might not even publish anything, and we might want to validate that no event was published. For example, if the stock didn't change, the service won't send a stock changed event. Typically, we use a timeout but can increase the time the tests run significantly. An alternative can be sending a testing purpose event at the end of processing the order event. When the service finishes the processing, it publishes an event signaling the finish of the operation, whether it sent a stock changed event or not. Changing the code to be testable is a common practice in unit tests; however, in this case, it adds some complexity and adds code that is only used by the tests.

An interesting alternative is to monitor the order created event queue and wait for the inventory service consumer to acknowledge the successful processing of the event (e.g., in Kafka, wait for the consumer group to run out of lag). This way, we can be sure the service ended the processing without introducing additional developments in the service.

Guaranteeing the tests are autonomous has high synergy with applying a container approach to dependencies. We can run the database and message broker containers, set up the initial schema and data, run the tests, and in the end clean everything by disposing of the containers. This

way, we guarantee no garbage data remains in a quality environment or data from older test runs don't end up affecting the current tests.

Mocking or stubbing the synchronous requests, like the dependency to the location service, provides the decoupling of the service with the rest of the ecosystem and enables autonomous releases. We no longer need to worry about configuring services out of the scope of the service we are releasing, and overall the tests enjoy higher stability and consistency. However, mocks are heavily biased, especially when defined only by the consuming team, and releases are susceptible to defects in the interface interaction between both services. No guarantee is made if the location service makes a change in its contract or behavior. A new release of the location service can be deployed live with its tests passing and break every other consumer.

To overcome this limitation, we can complement these tests with contract tests, consumer-driven contracts, and production tests. We will further these subjects in the remainder of the chapter. Another alternative is to use end-to-end tests, but they suffer from even more constraining issues. Overall half-breed integration tests are a great complement to component and unit tests, especially when we can't use extended component tests. They can provide broader scope validations of the behavior of the service along with the correct usage of its external dependencies.

10.1.6 End-to-End Tests

End-to-end tests validate the system as a whole, typically its correctness according to broad business rules. They often go well beyond the scope of a single service; they validate the business flow across multiple services and provide high-level, technology-agnostic, and business-facing validations. They are the type of tests that give the most confidence in the system since they cover a large ground. However, they have a mismatch with the distributed event-driven mindset, often have an overwhelming cost, and maintaining them is often extremely challenging.

Except for integration tests, all test categories we discussed until now are in the service's scope; if we include any of those tests in the service release pipeline, the service would only depend on itself. On the other hand, end-to-end tests include the

collaboration with several services; if we are to release any service, we need to run the end-to-end tests. Figure [10-7](#) illustrates the end-to-end tests scope in the example we discussed until now.

Figure 10-7 End-to-end tests scope

Not only the tests include the inventory service, its database, and message broker but also the location, order, and product catalog service. It would likely also include other upstream and downstream services, for example, the UI where the user submitted the order and the shipping and pricing services.

Releases

Since end-to-end tests target the entire microservice ecosystem, we must run the tests every time we release any service within the test's scope. These kinds of tests are slow by nature; they typically involve several services with several dependencies, ranging from databases, message brokers, distributed caches, etc. When the tests run into an issue, for example, if we are doing a release of the location service and the tests are failing, every other release of the order, inventory, and product catalog service would have to wait for the fix.

Remember when we discussed in Chapters [3](#) and [4](#) how services must be independent and autonomous, how event-driven architectures potentiate this characteristic and facilitate a truly continuous delivery mindset? This can easily escalate to a step back in that mindset; a service is no longer autonomously released; it depends on the successful implementation of several other services. It also leads to complex lockstep releases and quickly escalates to releasing everything in a single moment, which is what we need to avoid by adopting an event-driven architecture.

Data and Environments

Another substantially cumbersome challenge of end-to-end tests is state management. Services without state are easier to handle in an end-to-end context since no data bootstrap is needed. However, many services have

data and need to have data initialized in order to run test cases. This is a common challenge; we discussed it before when we discussed integration tests, but in that case, the service only had to manage its data. In end-to-end tests, each service has to have relevant data, and worse, all data need to be consistent with each other.

In an integration test, the tests may initialize test data and run the tests against that data set. With end-to-end tests, the services not only need to initialize their state but need to make sure it is coherent with every other service. For example, it's not enough to set up test stock data in the inventory service, we also need to guarantee the external identifiers like the products we assign stock to must be the same as the ones in the order and product catalog service. The tests aren't in charge of only one service but require an orchestration across all services in the test scope just to set up the conditions for the test to run. This necessity often requires intrinsic coupling between the overall flow and business logic and the tests. This makes the tests exceptionally brittle and susceptible to changing requirements. Often tests become more complex than the functionality under test.

The complexity of distributed architectures often makes teams build an integrated quality environment where the end-to-end tests run. Maintaining this environment is often complex and time-consuming. We need to guarantee the services are updated every time a release is deployed live. It can be done automatically, but often the intricacies of each service, their configurations, lack of discipline managing the data stores and message brokers end up breaking the quality environment frequently. It quickly leads to having individuals or full teams dedicated to maintaining the environment stable.

Another downside of the data bootstrap in quality environments is leftover data from previous test runs. Message brokers and data stores tend to be a dumping ground for test data that often is responsible for causing the tests to fail with false positives. It also further deviates the quality environment from production. A good approach to this challenge is to build a system on demand, deploying containerized versions of the

services and dependencies to an environment, and clean everything up in the end. However, the tests may take longer to run.

Governance

Since end-to-end tests span several services, it makes sense to run the tests in the release of every service in the scope of the test. But that will likely mean several teams might be involved in the development and ownership of those tests. Deciding who has the direct ownership of the tests can be troublesome; we can assign it to a team that is highly involved in the overall flow; for example, assign the order fulfillment test to the team that owns the order service. However, it can get tricky to manage when those tests fail because the stock wasn't reflected on the product catalog service. That team has limited context of that service; requesting assistance from the respective team might be the way to go but harder to effectively do if they don't have skin in the game.

Another alternative people sometimes take is to have a dedicated team to develop those tests. This can be even more difficult to manage; that team has limited context about the effective implementation of the services and makes the teams that own the services even more detached from the whole process. This team often becomes a gatekeeper for releases making teams losing even more autonomy.

A better approach can be to have the tests with shared ownership; each team is responsible to contribute and develop the tests. If the tests fail, the team who is releasing the feature is the driver to find the issue; if it involves other services, the corresponding teams must help understand the issue; communication is key. A possible downside of this approach is the lack of drive to resolve flaky tests; people trying to make the tests pass for their release even if they have to retry the tests a few times ("a dog with two owners dies of hunger"). However, it is a preferable approach to deal with the test's governance.

Approaches to Mitigate the End-to-End Tests Challenges

These are a few examples of the challenges of end-to-end tests. The context of end-to-end tests often varies from company to company; however, here is some advice to

address these concerns:

- Do you really need end-to-end tests? Distributed architectures require the correct approaches and techniques to quality assurance rather than trying to apply old ones to a new context. We will detail alternatives to them in this chapter and make an informed decision whether they are worth the cost. Of course each company has its own context and risk tradeoffs, so will yours, but ask yourself how many end-to-end tests you need and if (or how) you can replace them. Often it's not about losing quality but making quality sustainable.
- Perhaps you don't need to run end-to-end tests on every release but rather on the end of larger projects or technical revamps. An approach I saw used to some success in larger projects, is to have each service deliver its features live, but disabled. Before enabling the functionality, the teams ran an end-to-end test to guarantee there are no issues in the high-level flow; if any is found, the team would develop a component or unit test to guarantee it doesn't happen again. Typically, large projects or major restructurings benefit from end-to-end tests; using them on an ad hoc basis and guaranteeing continuous quality through other means might be a useful approach.
- Perhaps you don't need to validate the whole end-to-end flow. Often a segment of the whole flow is more critical than the remaining flow; validating that segment has more value than exercising the complete flow. Measure stability over test coverage and make informed decisions.
- Using a containerized environment that bootstraps, runs the test suite, and cleans everything up can be a good approach to difficult state management issues later on. It can also be a stepping stone for running the tests locally in the developers' computers instead of using a shared quality environment. Running the tests locally can also be a good approach to avoid lockstep releases where an issue in one service blocks the releases of the other services. However, even when adopting infrastructure-as-code practices, this approach gets trickier with complex flows. Not only bootstrapping the whole constellation of services can be extremely confusing and time-consuming, but also maintaining it requires substantial effort. With high numbers of services, it might be hard to have a machine with resources capable of

running every service and its dependencies. Focusing on a critical part of the end-to-end flow can be a reasonable option.

- Guarantee autonomous management of data and event schemas, event streams, and data stores. You must strive for guaranteeing the setup of the environment is automatic. Every change in the schema by the teams must have an associated script or process; the process to deal with event's breaking changes must be accounted for in the quality environments in an automatic way, avoiding manual corrections in the quality environments.
- If you do need end-to-end tests, guarantee you only have a very limited amount. Most validations can be done by combining the other types of tests (unit, component, and integration) and highly contain the responsibility of end-to-end tests. Use them only for critical high-level flows and focus on clear business processes.

End-to-end tests can be applied successfully to small architectures with a very limited number of components which likely only one team manages. If this is your case, it might make sense to have them; however, they tend to get worse as we add more components and the architecture evolves. It gets even worse when we lose autonomy to release functionality, which typically is severely aggravated when multiple teams are involved in the same tests.

10.2 Applying Contract Tests and Consumer-Driven Contracts to Event-Driven

Contract tests are often associated with schema validations, like breaking, forward, or backward changes, as we discussed in Chapter 8. But they go further than that; they validate the correctness of a producing service against the expectation of a consumer. Automatic schema validation is pivotal in event-driven services and can be provided by schema registries (like Avro in Kafka⁸). Contract tests go beyond agnostic schema validation rules; they validate the inputs and outputs of producing services against the needs of consumers. This section will discuss their scope in event-driven architectures and how we can combine them with consumer-driven contracts.

Contract tests typically sit in the boundary of a service with an external interface. Their focus is the compliance of the producing service with what consumers expect. Figure [10-8](#) illustrates the scope of contract tests in the same example we discussed until now.

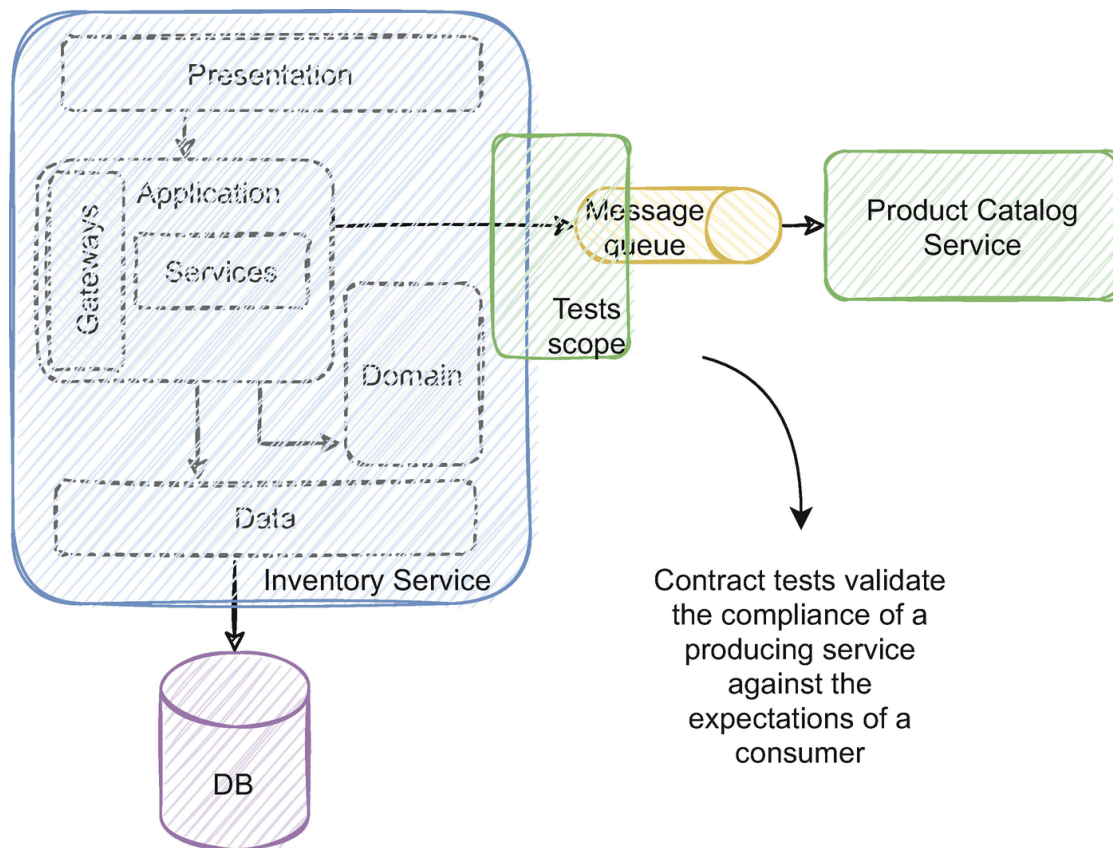


Figure 10-8 Contract tests scope

In this case, the product catalog service consumes stock events produced by the inventory service. An interesting contract test we could add to the inventory service is guaranteeing every information needed by the product catalog service is being published. We can use mock or in-memory replacements of the message broker to achieve better performance and stability. However, many use cases benefit from having a dependency to a message broker due to serialization and compatibility concerns. The tests can guarantee the consumer is able to read the message by consuming the events in the same way, using the same schema, serialization protocol, headers, etc. For example, sometimes different consumers have different serialization protocols for different event streams, and it's easy to mess up the configuration. This way, we can have a broader guarantee that the consumer will be able to read the message.

It can also guarantee some business logic the consumer is relying upon. Some events might have data in some fields in certain situations. For example, if we have multiple steps in the creation of a product, the first step might fill only the mandatory data, and the remainder of the information will be enriched later in the process; for example, we can create the product and add stock later. However, stock requires the size to exist in the product; for example, we add stock to an XS size. In this case, the product service might expect that all stock to be associated with a product size. A possible validation could be when the stock is greater than zero, the size is also published. These kinds of validations go further than direct schema validations; they guarantee an expected behavior in the producer, which if broken will cause issues in the downstream components.

Until now, we discussed how the team managing the inventory service could unilaterally add contract tests to the service. A valuable iteration of contract tests is to align them with consumers, typically referred to as consumer-driven contracts. In Chapter [8](#), we discussed the importance of consumer collaboration in the schema definition. This approach takes that collaboration one step further; the team managing the producing services has the mission to understand and guarantee the expectations of the consumer services in direct cooperation with the consumer teams. Consumers can even implement the tests in the producing service to guarantee their expectations. Figure [10-9](#) illustrates this scenario.

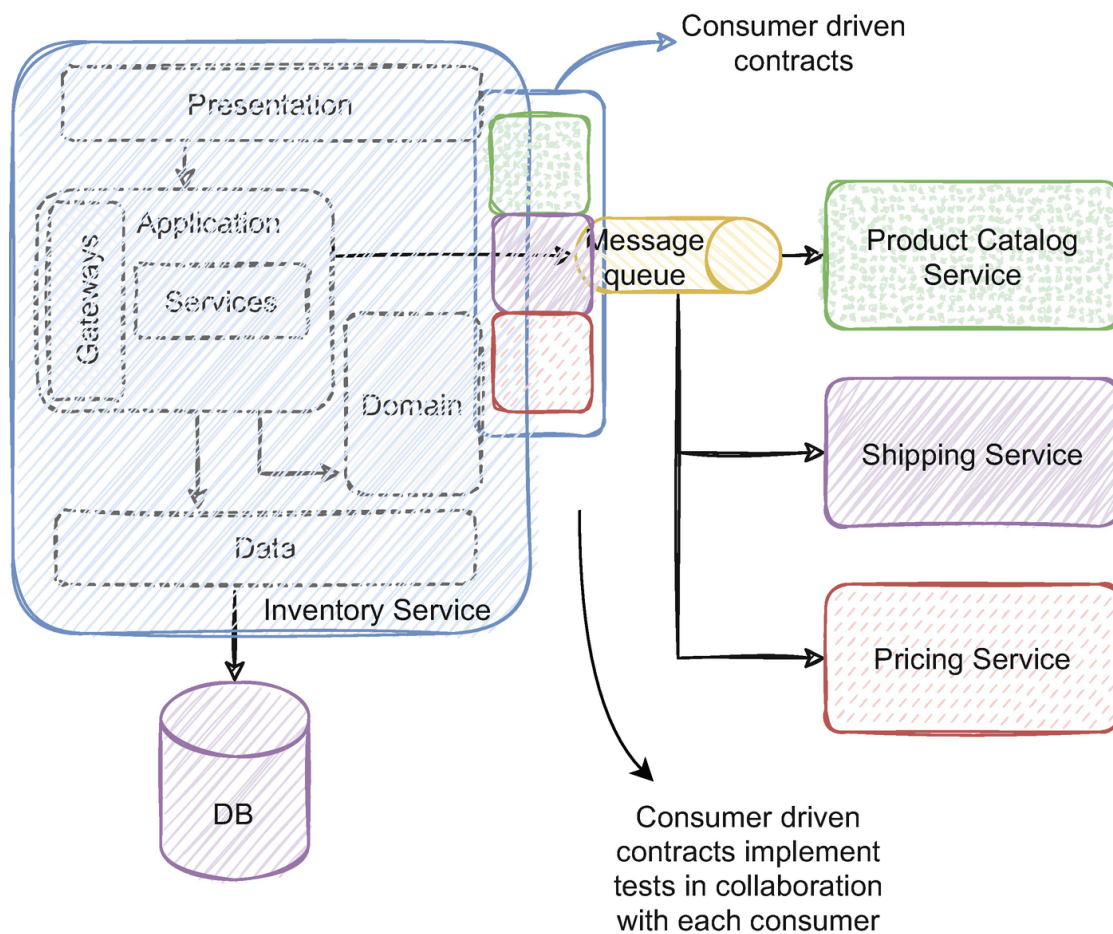


Figure 10-9 Inventory service consumer-driven contracts with multiple consumers

If we run the contract tests for each release of the inventory service, we have greater confidence that the functionality we are releasing won't break existing consumers. They also encourage communication between both teams. Communication is pivotal in guaranteeing quality and understanding the needs of consumers; coding them down in the form of tests is a great way to guarantee their expectations are being met. Ownership of a correct message flow in every release is distributed to the different teams involved, which usually helps to guarantee everyone is on the same page.

Business processes are often choreographed with a series of events in event-driven architectures. Applying contract tests and consumer-driven contracts to every component in the flow can be a way to guarantee more scalable validations of the end-to-end flow.

10.3 Test Categorization and Purpose

The types of tests we discussed until now have different purposes and scope. Categorizing the different kinds of tests can help make sense on how and when to apply them. Brian Marick defined four quadrants for test categorization⁹ that helped to clear a lot of misunderstandings when discussing the various categories. We can think of it by having tests focused on verifying what is expected against finding unforeseen corner cases or unexpected use cases. In the original Marick's categorization, those test categories can also be more oriented to business or technology, as illustrated in Figure 10-10.

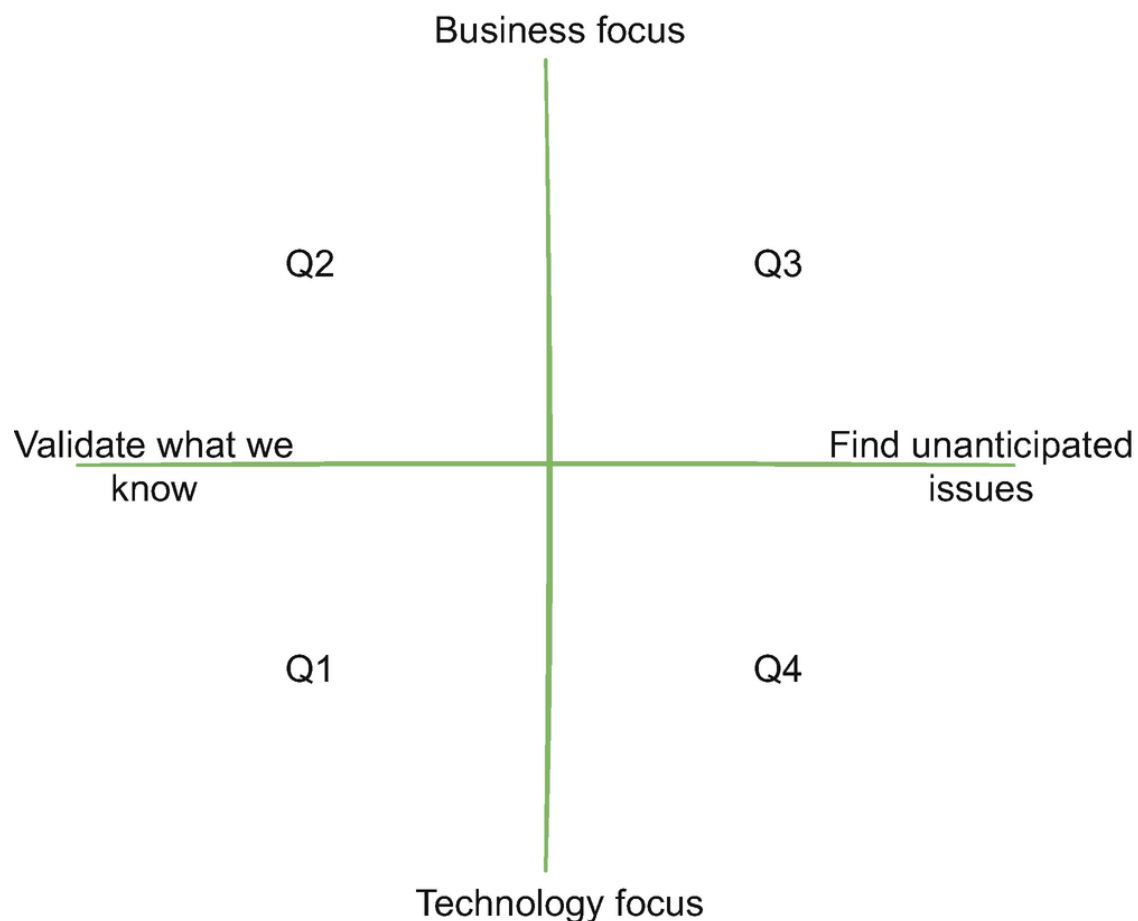


Figure 10-10 Test categories divided by their focus and purpose

Using this categorization, we can find some examples for each quadrant:

- Q1: Fully automated tests oriented to verifying what we developed works as we expect it to work. Can be unit, component, or integration tests.
- Q2: Automated (or manual) tests to verify if we developed the correct functionality as defined by the business. Typically, functional and acceptance tests, including business validations of the application.
- Q3: Manual exploratory tests dedicated to finding issues with the application. These tests are creative in nature and are oriented to people

interacting with the application in unexpected ways.

- Q4: Tests to verify the non-functional requirements and technical limits of the application. Can be performance and load tests, security validations, and other quality attributes like availability, correctness, or reliability. It usually relies on automation or specialized tools.

Although this categorization makes a lot of sense, some areas can be subjective. For example, if an acceptance test finds an issue with the application, we could, if possible and if small scoped enough, develop an automated unit test to guarantee the issue doesn't happen again. Unit tests can be used to guarantee the soundness of business rules as long as they are small scoped enough (typically in the context of a specific part of the code like a function), although that unit test would now be technology-focused rather than business-focused.

Quality assurance practices have come a long way from the extensive manual testing plans. It's actually pretty obvious the advantages of not having a single person or a team of individuals performing a repetitive set of actions every time there's a release (there's even the tale of a test engineer who automated his job for six years without the company noticing¹⁰). Test automation isn't just smart, it becomes a necessary condition to successfully scale a distributed architecture. If you think about it, extensive manual regression tests might be an option in a single deployable component like a monolith, but when you have to operate dozens or even hundreds of autonomous components with several daily releases, it quickly becomes unviable to scale the number of manual testers to cope with the growing number of releases.

However, it doesn't mean there's no place for manual testing. Repetitive tasks should and need to be automated, but there's a rightful place for curious and creative ways to break the application. Automating repetitive tasks frees quality engineers to do much more meaningful and creative work of interacting with the application in unforeseen and unexpected ways through, for example, exploratory tests. Manual testing can also be relevant for use cases where we simply can't automate due to the effort or cost involved; I mostly saw this happening with extensively large legacy applications. In event-driven architectures due to the component's

high decoupling and the large number of independent components, a strong test automation culture is advisable.

All the categories discussed in this and the previous sections are in the context of pre-production testing. As we will discuss at the end of the chapter, production testing approaches that we will discuss in this chapter can be categorized into the different quadrants of Figure [10-10](#). Production testing is an increasingly adopted practice that helps overcome the challenges of quality assurance in complex distributed systems and can be extremely valuable as a complement to traditional pre-production approaches.

10.4 End-to-End Quality Without End-to-End Tests

For years, end-to-end testing was the quintessential bastion of quality and a beacon of trust in the tempestuous releases of new functionality. In truth, they can be a valuable quality assurance strategy in monolithic applications. However, as we discussed in Section [10.1](#), they have serious downsides when applied to fully distributed microservice architecture. Event-driven architectures nurture several fully decoupled, independent, and scalable components, and as we will see in this section, end-to-end tests have strong dissonance with the event-driven approach. The approach to quality assurance requires different strategies and a shift in the traditional mindset.

Despite the adoption of microservice architectures, many companies continue to have an approach to quality the same way they always did, by deploying all functionality in an environment and running the test suite against the whole system. I always found it peculiar how we faced the limits of monolithic databases with ACID properties by moving to highly distributed architectures, featuring high numbers of components asynchronously communicating with each other, often with different database technologies with weaker consistency guarantees, and still managed to maintain the consistency needs for the business to run. But our approach to quality is still deploying and running tests against every single functionality in a single place. We were able to develop patterns to deal with something as complex as consistency and state management across

distributed components, but why are we unable to have a similar approach to quality assurance? Why do we need to apply the old quality assurance approaches to a fundamentally new context?

In the same way, we don't lock every single microservice database to achieve atomicity; we shouldn't deploy every single component and block each service's release to guarantee quality. The distributed nature of event-driven architectures requires an approach to quality and quality assurance strategies that preserve boundaries and independent releases. One of the most valuable characteristics of event-driven architectures is the decoupled nature of its components. This enables the architecture to be truly evolutionary; features can be implemented separately from each other; services are autonomous and independently released. It allows for teams to truly own their services and do their work without the shackles of dependencies, approvals, or gatekeeping processes. End-to-end testing is a deliberate and confident step in the opposite direction. We face all the challenges we discussed throughout this book to achieve this clear segregation, single responsibility focus, well-defined domains, autonomy, and decoupling and, many times, to throw it away in the quality assurance process.

Adopting a truly decoupled and distributed architecture requires the correct approaches to quality assurance, instead of forcing traditional techniques to a radically different environment. One way to do this is to combine some of the strategies we discussed in Section [10.1](#). We can approach end-to-end quality as the sum of the quality of each component, as illustrated in Figure [10-11](#).

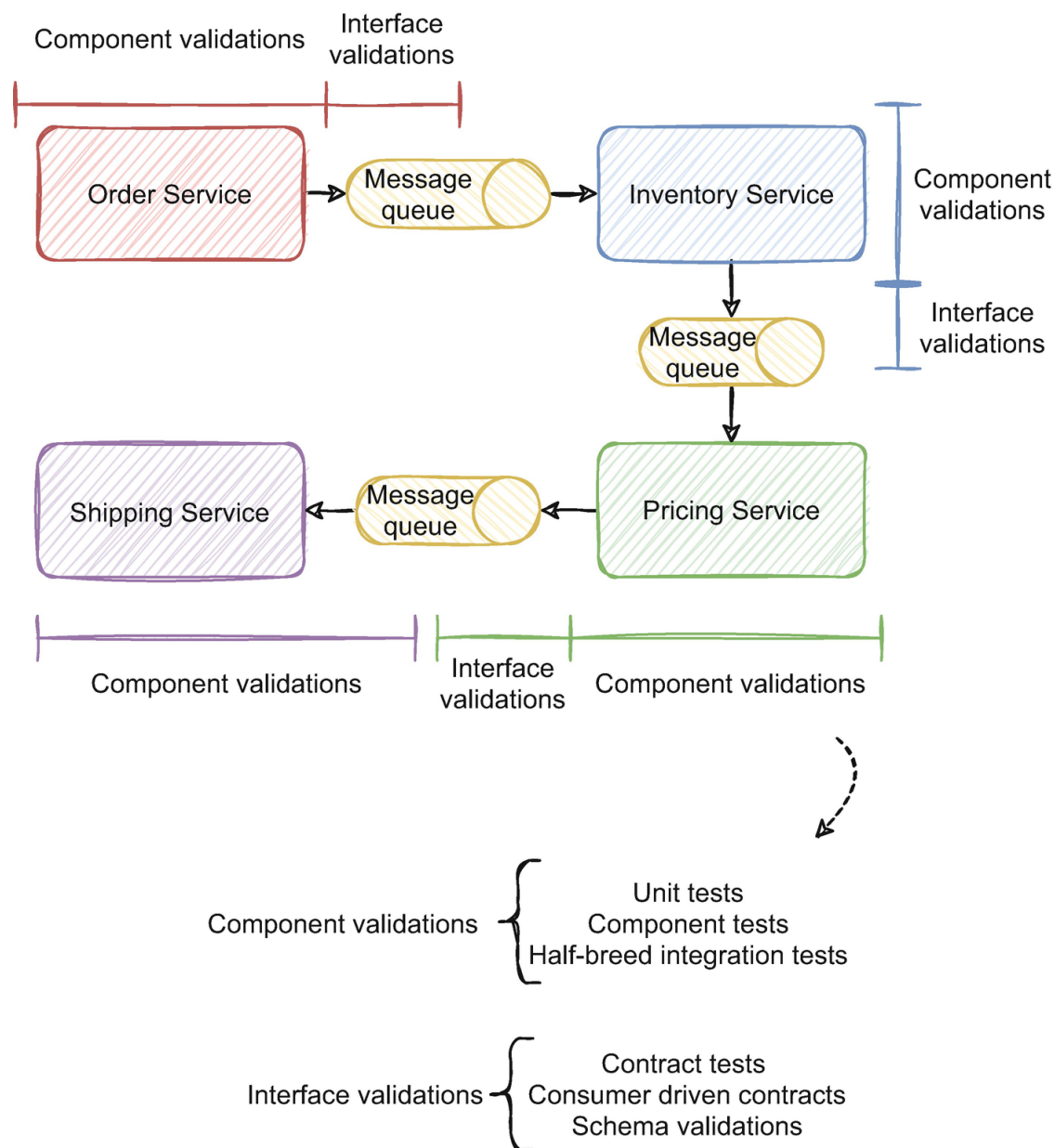


Figure 10-11 End-to-end quality as the combination of quality assurance practices in each component

Each component needs to guarantee its quality independently, through unit, component, and half-breed integration tests. Integration tests across several services were intentionally left out since they suffer from many of the same downsides as end-to-end tests, but at a smaller scale. Each service must guarantee its correctness through its own tests, including its part in the end-to-end flow. It is pivotal that each team has a higher-level vision of the larger flow in order to understand and implement the validations of their components (this builds on the concepts we discussed in Chapter 4). The compatibility with other services is guaranteed through schema validations, contract tests, and consumer-driven contracts. Not only do they guarantee there are no unexpected changes in the event's schema but also guarantee the expectations and behavior of downstream

services are maintained. This is the situation where the sum of the parts is greater than the whole.

Typically, failures in the end-to-end flow map to one or a set of components, which can validate that condition in the scope of only the corresponding components. However, this approach requires close communication and alignment with each team, which consumer-driven contracts intend to tackle, at least partially.

I wouldn't say this approach is failproof, it isn't; however, neither are end-to-end tests. Maintaining a copy of the live environment is extremely expensive and time-consuming, and the result is a poor reproduction of the live environment at best. Production tends to be highly dynamic with differing usages, throughputs, and conditions that vary through time. An environment that only runs the tests doesn't really stand near to the live conditions. These environments often use a small number of machines with resources far smaller than production (to contain costs) but fail considerably in replicating the performance and concurrent conditions of live environments. They also typically stand in the same network host or in network conditions that minimize or eliminate several network failures like network partitions. They often only have a subset of the production data (if any) which can't really reproduce all the possibilities of data existing in production, for example, event streams covering every current or past use case. Configurations, network topologies, and connectivity conditions are often replicated but without any guarantee they remain the same as the live environment. After all, how many times did we run end-to-end tests only to find there were still issues after releasing the functionality?

End-to-end quality as the sum of its parts doesn't guarantee there won't be issues either. Would you be able to write a test for every single condition a service can fail? Probably not. Test suites excel in validating the known failures of the system. End-to-end tests are just one approach, perhaps valuable in monolithic applications, but extremely inadequate in distributed systems.

So, am I saying the only way is to live with the problem? Not at all. Pre-production quality assurance techniques can be complemented with test-

ing in production approaches to mitigate many of these issues, which we will discuss in the next section.

10.5 Testing in Production

I know what you're thinking, "I don't always test my code, but when I do, I do it in production." Well, I'm pretty sure you probably have your own tales of testing in production the same way as I have, but that's not quite it. Testing in production has become increasingly relevant in the context of microservice distributed architectures as a way to complement pre-production tests or even as the only feasible way to provide comprehensive end-to-end quality assurance guarantees. This section will discuss those approaches and how we can use them in event-driven architectures.

As we discussed in the previous section, pre-production tests can have limited coverage of the possible failure modes of a complex distributed system. It can become even more challenging when usage patterns change over time. Trying to cover every possible failure mode often requires such a complex testing logic and infrastructure that can even match the system under test. Besides being limited, leaving the burden of end-to-end quality assurance to exclusively pre-production strategies will invariably and substantially delay the release and feedback cycles. With the right tools and practices, we can use the production environment to increase our confidence in our quality and release process in a sustainable and comprehensive way, instead of trying to use a poor imitation of the live environment.

The most challenging aspect of implementing this practice is often getting people on board with the approaches. People used to the traditional testing practices and oblivious to the challenges of distributed architectures often frown at the very suggestion of testing in production. Sometimes we put unmeasured faith in automated testing, perhaps the same way as we did in dedicated quality teams that did manual regression testing in an era before automated tests, without honestly facing the viability of alternatives. The live environment is often looked at as a pristine forbidden artifact that shouldn't be touched (sometimes with good reason),

but we should also consider that the correct approaches can greatly benefit quality assurance processes without endangering the live conditions (Cindy Sridharan has a great series of articles that further these topics that I recommend¹¹). To be fair, health checks to services deployed in the live environment are a way to test in production. Often features are behind feature toggles, or we release functionality to only a segment of users; all of this can be seen as testing in production. Business validations occurring behind testing or sandbox users aren't unheard of. Testing in production might seem radical, but often they already are part of our processes, embracing them as what they really are; a powerful tool to fully achieve end-to-end quality can provide a more sustainable and transparent approach to quality assurance.

10.5.1 Shadowing

A common challenge of pre-production testing is the diversity, quantity, and accuracy of the data. Shadowing (also known as mirroring) is a strategy to expose new releases to production data without impacting the user. It can be extremely useful to guarantee new releases are able to interact successfully with real, real-time requests.

We often use mock data or generate data for our tests. There is a range of tools to generate random data based on given contracts (e.g., AutoFixture), which simplifies the data generation process. Despite its usefulness, generated data is random or manually set up, which always struggles to represent a comprehensive sample of the real production data. Random data can easily generate scenarios that don't make sense and even cause instability in the test suite. For example, if we are generating a stock changed event, the tool can easily generate random values for product identifiers with unexpected size. Although it might be useful to test those scenarios, a test like that might fail in one run and pass in the next. It is better to test deliberate scenarios than generating brittle tests through random data generation, thus the need to carefully set up the random data. Random data also tends to deviate considerably from real production traffic, and even when they are accurate, they hardly mirror the variety of scenarios we find in production.

Mocked or stubbed data suffer from similar challenges. They are also particularly susceptible to bias; we often set up mocked data with what we are expecting to receive. What we receive and what actually exists in production can be very distinct. In production, there are also edge cases with data that we don't expect, and thus we don't create test cases for those use cases. This is often a source of issues with teams with a high test automation culture. Unfortunately, when this happens, we can only detect it in production, despite the high coverage we think our tests have. As an alternative approach, we can also copy data from production. But typically we copy only a sample of data, which differs greatly in quantity and might not have samples for every use case.

With shadowing, we can help prevent these issues by replaying current production traffic to new releases. Let's work through a use case; imagine we are deploying a new version of the inventory service and its consuming order created events. Figure [10-12](#) illustrates this scenario.

Figure 10-12 Shadowing in an event-driven service

In this example, we have the inventory service with three instances in the live environment and a new version of the service we want to deploy. Before actually installing live and replacing the existing instances with the new version, the new release consumes concurrently the same events the live instances are consuming. For example, if we are using Kafka, we can simply have a separate configuration for the pre-production environment and use a different consumer group configuration for the new release. Since the new version will have a consumer group and the three instances will share a different one, the new version will consume the same events the three instances are consuming.

It is important to notice the new release is deployed in the live environment the same as a live instance and has access to all its resources, but with different configurations. The main goal is guaranteeing the service is able to correctly process live traffic with the new features. So the service will process the live event stream, and we can automatically detect changes in error rate and have automatic validations of the service behavior. We also need to guarantee the pre-production instance doesn't

effectively change the live state. The new instance will consume the messages, apply its business logic, but shouldn't save state in the database or publish events to the event stream. We can implement that by having different configurations that inject fake message broker and database implementations.

An interesting metric to monitor could be the throughput of the service. But since we are using fake implementations of the external dependencies, it wouldn't be accurate. We also wouldn't be able to validate the implementation against a real database or message broker. We can further elaborate on this pattern by having a separate data structure or event queue with the pre-production data. Figure [10-13](#) illustrates this scenario.

Figure 10-13 Shadowing in an event-driven service with live dependencies interaction

In this example, the new release reads from the live event stream and the live database but writes to different structures than the live instances. For example, the new release can publish to a staging queue inside the same broker and write to a staging table inside the same database. This guarantees the new release is able to write successfully to the external dependencies. We can also monitor relevant deviations in the new release's throughput since it should be roughly the same as the live instances (lag will be different and likely to increase in pre-production since only one instance is processing against three). We can also run automatic validations to compare both event streams and detect deviations. If we are generating unexpected events or events with incorrect data, when compared to the real live event stream, it would be an indication to invalidate the release and automatically fail the deploy. We can do the same with the database's state.

Shadowing can also be applied to synchronous requests, albeit with the support of an external tool like a proxy. The proxy can duplicate requests to a pre-production instance similar to what we discussed with the inventory service. Figure [10-14](#) illustrates this scenario.

Figure 10-14 Shadowing in with synchronous requests

In this example, the inventory service exposes an API to both writes and reads. With the help of the proxy, we can duplicate a segment of the live traffic to the pre-production instance. Some tools support this functionality out of the box; for example, HAProxy supports this kind of functionality seamlessly.¹² Reads don't have any impact; we can simply redirect a portion of the reads to the new version and validate its results against the live instances. If we want to validate writes, we can take an approach similar to what we discussed with purely event-driven services by writing to a staging structure and validating both.

Shadowing is exceptionally useful by allowing us to test new functionalities against real-time, live traffic without impacting the user. The downside is we have a new instance consuming from live resources; although they usually can (and should) handle the load of an additional instance, we need to guarantee the dependencies (like the broker and database) are prepared for it. Either way, it is an especially valuable and simple technique to guarantee new releases won't run into issues when exposed to live traffic, without the limitations of sampling data from production to quality environments.

10.5.2 Canaries

A popular approach to production validation is canaries. Canary involves exposing only a segment of end users to new functionality, instead of making it available to everyone instantly. Unlike shadowing, this approach only minimizes risk; end users are still exposed to the new functionality, but on a smaller scale. This section will detail the several approaches we can take in implementing canaries.

One possible approach is to deploy one instance with the new version and add it to the pool of production instances. Since traffic is distributed across all instances, the new version will receive a part of the live traffic where the already existing instances will receive the remaining, as illustrated in Figure [10-15](#).

Figure 10-15 Canarying by sharing the live instances pool

Typically, traffic is approximately equal across all instances. In this example, since there were three instances in live and we added the new one to the instance pool, the new instance will receive only one fourth of the live traffic. In case there's an issue, only one fourth of the traffic will be impacted and we can roll back the release.

Alternative approaches can forward only a selected segment of traffic to the new version. We can first direct only internal traffic to the new instance; for example, the internal teams can first interact with the new version before rolling it out for the entire public. This approach is easier with synchronous requests and a proxy that can selectively deliver requests to specific instances using a location or header. With event-driven services, it can be harder since services would have to selectively process or ignore events that weren't directed to them or implement a custom routing algorithm. A further iteration of this approach is to divide traffic by relevant business information, like country or user segment. This approach is even more relevant with a lower number of instances (in this case, 25% of the traffic would be processed with an error).

The main issue with canaries is when there are issues, end users will be faced with an error. In event-driven services, it also means an invalid build was temporarily running against live data. Roll backing no longer completely solves the issue since downstream event streams or the database might be corrupted. An approach to mitigate this is what we discussed in Chapter [7](#) by rewinding the queue to the state before the deployment. A further challenge is to deal with database or event schema changes which require a more careful approach. A possible solution is to avoid breaking changes and do forward and backward schema changes instead of deploying the breaking change in a single deployment, as we discussed in Chapter [8](#). However, it adds complexity in favor of security and stability.

10.5.3 Feature Flagging

A common way to implement testing in production is through feature flagging, where new features are deployed but disabled by a feature flag. We can test the functionality's correctness and compliance with non-busi-

ness requirements like throughput and response time and activate it when we are confident the feature works as expected. Deactivating the feature is also straightforward since it only involves switching a configuration.

Feature flagging varies in scope; it can be all or nothing where we either activate or deactivate the functionality. Or it can be more intricate processes where we roll out new functionality to increasing amounts of end users for a given time period, similarly to canaries. There are also several types of toggles; they can range from toggling business functionality and non-functional requirements to security and experimentation features.

The concept is straightforward; we have a set of configurations that activate functionality depending on whether they are active or not. They can be more elaborate configurations when we only want to activate a segment of live traffic, for example, a configuration with all the countries we want to activate the feature to at that time. When processing an event, we decide which path to follow based on the event's data or headers; if it is an event from a configured country, we activate the new features.

A common pitfall is the complexity some toggles have and how they sometimes become entwined with the core business logic. Following especially closely the single responsibility principles and dependency inversion can help decouple the toggle logic with the reminder service's functionality. It is also important to have a strict rule to clean old flags when they are no longer needed. The build-up of several feature toggles through time can make the service needlessly complex. The added complexity will also reflect in the test suite since it should test both paths while the toggle exists. Thus, a strong discipline to refactor and clean old toggles is important. Although feature flagging is often a simple and straightforward strategy, some toggles extend through long periods of time, making extensive use of this approach more complex than it needs to be. However, with the right code refactoring concerns, it is a simple and incremental approach to expose functionality to live data.

10.5.4 Production Automated Testing

A further development of production testing practices is to run automated tests directly against production instances. This approach is arguably the most radical and requires strong alarmistic, monitoring, metrics gathering and control over the release process. With the right approach, combined with the traditional pre-production strategies, it can provide the most comprehensive coverage of the test cases that we can't or are too costly to set up in pre-production environments.

To be fair, this approach is perhaps one of the most mature stages a company can implement. A common evolution path is typically to start with traditional end-to-end testing and automated component tests and then implementing strong logs collection, metrics, and tracing. Then move to more straightforward approaches like feature flagging and canaries. Shadowing could arguably be the next logical step, while production automated testing is the very last.

The approach involves having a test suite that runs directly on new releases on production instances. This way, we can have test cases that test the full integration with the service dependencies. If the tests pass, we are sure there are no unforeseen cases that weren't picked up in the pre-production tests. Let's work through the use case of the inventory service we discussed until now. The service receives events, processes them, and publishes stock changed events to downstream services, in this case, the shipping service, as illustrated in Figure [10-16](#).

Figure 10-16 Production automated testing example

The new release is deployed alongside the remaining inventory service live instances. The test suite publishes events to the test event queue which are processed by the new version. If the service publishes events, it will publish to the real event stream and, in this case, should include a header indicating they are test events. The shipping service will use that header to ignore those events. The interaction with the database gets trickier, reads are fine, but when handling writes, there's the concern of polluting the database with testing data. We can have a staging structure like we discussed in the shadowing strategy or ignore test data by signaling those records. An alternative could be having specific users or segments of data dedicated to testing, for example, sandbox clients that we

know are only for testing. Many companies already do some kind of business validations directly in live recurring to these kinds of clients; running automated tests against them instead of manual validations might not sound so far-reaching.

With this approach, we can validate that the interfaces with other services are working according to the expectations of the inventory service. All validations also occur in concurrency with the whole ecosystem instead of having only the tests running. We can also realistically validate non-functional requirements like response time or throughput. As we discussed, besides all the things that can go wrong with traditional quality assurance approaches, distributed systems have a myriad of failure modes that are infeasible to reproduce in quality environments. These types of approaches can be a valuable addition to assure new functionality works as expected.

10.6 Summary

- There are several quality assurance strategies we can implement in event-driven services. They can range in scope and quantity; combining them can be a valuable approach to guarantee comprehensive test coverage.
- Unit and component tests only depend on the service, are usually fast and stable, but have limited scope. Extended component tests can be a valuable way to validate the interaction with external dependencies but can be hard to find the right in-memory replacements.
- Half-breed integration tests can be a good compromise between including standard dependencies and other services. Standard dependencies like message brokers or databases are typically easier to set up and are more stable than other services in the architecture.
- Integration and end-to-end tests are extremely challenging to implement consistently. They also quickly restrict the release process and the team's autonomy. There are approaches we can take to minimize these challenges, but they will always be costly to maintain. A more sustainable approach is to combine pre-production with testing in production approaches.

- Contract testing and consumer-driven contracts are a great way to validate the interfaces with other services and encourage collaboration and communication with both the producing and consumer teams.
- An alternative approach to full end-to-end tests in event-driven services is to combine pre-production approaches like unit, component, and half-breed integration tests with interface validations like schema validations, contract tests, and consumer-driven contracts.
- Shadowing can be an excellent approach to validate new functionality with live traffic without impacting the end user.
- Canaries and feature flagging can be valuable to gradually expose new functionality to increasing quantities of users. They can minimize the risk with new releases and limit the footprint of possible issues.
- One of the most mature approaches to quality assurance in event-driven architectures is automated testing in production. It can help in guaranteeing the coverage of failure modes pre-production approaches aren't able or are too costly to detect.

Footnotes

¹ More information in “EF Core In-Memory Database Provider,” October 27, 2016, <https://docs.microsoft.com/en-us/ef/core/providers/in-memory/?tabs=dotnet-core-cli>

² Full article by Jimmy Bogard, “Avoid In-Memory Databases for Tests,” March 18, 2020, <https://jimmybogard.com/avoid-in-memory-databases-for-tests/>

³ Home page at www.h2database.com/html/main.html

⁴ Full article by Ivan Ponomarev and John Roesler, “Testing Kafka Streams – A Deep Dive,” August 18, 2020, www.confluent.io/blog/testing-kafka-streams/

⁵ More details by Clinton Gormley, “Elasticsearch, the server,” August 30, 2016, www.elastic.co/pt/blog/elasticsearch-the-server

⁶ Home page at www.testcontainers.org/

⁷ Further details at www.testcontainers.org/modules/databases/

[8](https://docs.confluent.io/platform/current/schema-registry/schema-validation.html) More details in “Schema Validation on Confluent Server,” <https://docs.confluent.io/platform/current/schema-registry/schema-validation.html>

[9](http://www.exampler.com/old-blog/2003/08/22/#agile-testing-project-2) See article “Agile testing directions: tests and examples,” August 22, 2003, www.exampler.com/old-blog/2003/08/22/#agile-testing-project-2

[10](http://www.payscale.com/career-news/2016/05/programmer-fired-after-6-years-realizes-he-doesnt-know-how-to-code) News in www.payscale.com/career-news/2016/05/programmer-fired-after-6-years-realizes-he-doesnt-know-how-to-code

[11](https://copyconstruct.medium.com/testing-microservices-the-sane-way-9bb31d158c16) Full articles by Cindy Sridharan, “Testing Microservices, the sane way,” December 31, 2017, <https://copyconstruct.medium.com/testing-microservices-the-sane-way-9bb31d158c16>

[12](http://www.haproxy.com/blog/haproxy-traffic-mirroring-for-real-world-testing/) More details by Nick Ramirez, “HAProxy Traffic Mirroring for Real-world Testing,” July 23, 2019, www.haproxy.com/blog/haproxy-traffic-mirroring-for-real-world-testing/

[Support](#) [Sign Out](#)